



Figure 1: Graphics pipeline

as generation and transformation of texture coordinates can take place at this stage.

Next comes clipping, or the elimination of portions of geometry which don't need to be rendered on the screen and are an overhead. Once we have figured out which pixels need to be drawn to the screen, the varying values from the vertex shader will be interpolated across the surface and fed into the pixel shader. These pixels are first unpacked from one of a variety of formats into the proper number of components. The colour of each pixel is determined. Now, we can use this information to calculate lighting, read from textures, calculate transparency, and use some high level algorithms to determine the depth of the pixels in 3D space.

We now have all the information that we need to map the pixels onto the screen. We can determine the colour, depth and transparency information of every single pixel, if needed. It's now the GPU's job to decide whether or not to draw the new pixels on the top of pixels already on the screen. Note that, prior to this stage, various complexities in the pipeline can occur. If textures are encountered, they have to be handled; if fog calculations are necessary, they are to be addressed. We can also have a few stencil tests, depth-buffer tests, etc. Finally, the thoroughly processed pixel information, the 'fragment', is drawn into the appropriate buffer, where it finally advances to become a pixel and reaches its final resting place, the display screen.

Note that the above described 'fixed-pipeline' model is now obsolete on modern GPUs, which have grown faster and smarter. Instead, the stages of the pipeline, and in some cases the entire pipeline, are being replaced by programs called shaders. This was introduced from OpenGL version 2.1 onwards. Modern 3D graphics programs have vertex shaders, evaluation shaders (tessellation shaders), geometry shaders, fragment shaders and compute shaders. It's easy to write a small shader that mimics what the fixed-function pipeline does, but modern shaders have grown increasingly complex, and they do many things that were earlier impossible to do on the graphics card. Nonetheless, the fixed-function pipeline makes a good conceptual framework

on which to add variations, which is how many shaders are created.

Diving into OpenGL

As described, OpenGL is designed to be hardware-independent. In order to become streamlined on many different platforms, OpenGL specifications don't include creating a window, or defining a context and many other functions. Hence, we use external libraries to abstract this process. This helps us to make cross-platform applications and save the real essence of OpenGL.

The OpenGL states and buffers are collected by an abstract object, commonly called context. Initialising OpenGL is done by adding this context, the state machine that stores all the data that is required to render your application. When you close your application, the context is destroyed and everything is cleaned up.

The coding pattern these libraries follow is similar. We begin by specifying the properties of the window. The application will then initiate the event loop. Event handling includes mouse clicks, updating rendering states and drawing.

Here's how it looks:

```
#include <RequiredHeaders>

int main(){
    createWindow();
    createContext();

    while(windowIsOpen){
        while (event == newEvent()){
            handleThatEvent(event);
        }

        updateScene();

        drawRequiredGraphics();
        presentGraphics();
    }
    return 0;
}
```

Coming to which libraries to use with OpenGL, we have a lot of options: https://www.opengl.org/wiki/Related_toolkits_and_APIs.

Here are a few noteworthy ones:

- The OpenGL Utility Library (GLU) provides many of the modelling features, such as quadric surfaces and NURBS curves and surfaces. GLU is a standard part of every OpenGL implementation.
- For machines that use the X Window System, the 'OpenGL Extension to the X Window System (GLX)' is provided as an adjunct to OpenGL. The X Window System creates a hardware abstracted layer and provides